TELECOM_CHURN ANALYSIS.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAM Disk

Share

+ Code + Text

[1] 0s
```
# CHURN -
# IBM - OPEN SOURCE- TELECOM INDUSTRY
```

[2] 0s
```
# importing all librarires
```

[3] 1s
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

[4] 0s
```
# load the dataset
```

[5] 0s
```
df = pd.read_csv("/content/WA_Fn-UseC_-Telco-Customer-Churn.csv")
```

[6] 0s
```
df.head()
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity	...	DeviceProtection	TechSupport	StreamingTV
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	DSL	No	...	No	No	No
1	5575-GNVDE	Male	0	No	No	34	Yes	No	DSL	Yes	...	Yes	No	No
2	3668-QPYBK	Male	0	No	No	2	Yes	No	DSL	Yes	...	No	No	No
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	DSL	Yes	...	Yes	Yes	No
4	9237-HQITU	Female	0	No	No	2	Yes	No	Fiber optic	No	...	No	No	No

5 rows × 21 columns

[7] 0s
```
# as we seen the data is huge..
# so lets check for the columns
```

[8] 0s
```
df.columns
```
```
Index(['customerID', 'gender', 'SeniorCitizen', 'Partner', 'Dependents',
       'tenure', 'PhoneService', 'MultipleLines', 'InternetService',
       'OnlineSecurity', 'OnlineBackup', 'DeviceProtection', 'TechSupport',
       'StreamingTV', 'StreamingMovies', 'Contract', 'PaperlessBilling',
       'PaymentMethod', 'MonthlyCharges', 'TotalCharges', 'Churn'],
      dtype='object')
```

[9] 0s
```
# for total customers
```

[10] 0s
```
df.shape
```
```
(7043, 21)
```

[11] 0s
```
# the dataset contain 7043 customers with 21 unique features
```

[12] 0s
```
# each row represent the unique customer
```

[13] 0s
```
# data types - check for data types
```

[15] 0s
```
df.dtypes
```

	0
customerID	object
gender	object
SeniorCitizen	int64
Partner	object
Dependents	object
tenure	int64
PhoneService	object
MultipleLines	object
InternetService	object
OnlineSecurity	object
OnlineBackup	object
DeviceProtection	object
TechSupport	object
StreamingTV	object
StreamingMovies	object
Contract	object
PaperlessBilling	object

PaymentMethod	object
MonthlyCharges	float64
TotalCharges	object
Churn	object

dtype: object

[16] ✓ 0s
```
# convert total charges to numerical values
```

[18] ✓ 0s
```
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors= "coerce")
```

[19] ✓ 0s
```
df["TotalCharges"].dtypes
```
```
dtype('float64')
```

[20] ✓ 0s
```
# duplicated
```

[21] ✓ 0s
```
df.duplicated().sum()
```
```
np.int64(0)
```

[22] ✓ 0s
```
# missing values
```

[23] ✓ 0s
```
df.isnull().sum()
```

	0
customerID	0
gender	0
SeniorCitizen	0
Partner	0
Dependents	0
tenure	0
PhoneService	0
MultipleLines	0
InternetService	0
OnlineSecurity	0
OnlineBackup	0
DeviceProtection	0
TechSupport	0
StreamingTV	0
StreamingMovies	0
Contract	0
PaperlessBilling	0
PaymentMethod	0
MonthlyCharges	0
TotalCharges	11
Churn	0

dtype: int64

[24] ✓ 0s
```
# only one columns- TotalCharges have the missing values
```

[25] ✓ 0s
```
# total charges is in float
# so missing values replace with - median()
```

[26] ✓ 0s
```
df["TotalCharges"].median()
```
```
1397.475
```

[27] ✓ 0s
```
df["TotalCharges"].fillna(df["TotalCharges"].median() , inplace= True)
```
```
/tmp/ipykernel_594/2503773831.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the oper


  df["TotalCharges"].fillna(df["TotalCharges"].median() , inplace= True)
```

[28] ✓ 0s
```
df.isnull().sum()
```

	0
customerID	0
gender	0
SeniorCitizen	0
Partner	0
Dependents	0
tenure	0

PhoneService	0
MultipleLines	0
InternetService	0
OnlineSecurity	0
OnlineBackup	0
DeviceProtection	0
TechSupport	0
StreamingTV	0
StreamingMovies	0
Contract	0
PaperlessBilling	0
PaymentMethod	0
MonthlyCharges	0
TotalCharges	0
Churn	0

dtype: int64

[29] 0s
```
# categorical columns - unique values
```

[30] 0s
```
df["gender"].unique()
```
```
array(['Female', 'Male'], dtype=object)
```

[31] 0s
```
df.columns
```
```
Index(['customerID', 'gender', 'SeniorCitizen', 'Partner', 'Dependents',
       'tenure', 'PhoneService', 'MultipleLines', 'InternetService',
       'OnlineSecurity', 'OnlineBackup', 'DeviceProtection', 'TechSupport',
       'StreamingTV', 'StreamingMovies', 'Contract', 'PaperlessBilling',
       'PaymentMethod', 'MonthlyCharges', 'TotalCharges', 'Churn'],
      dtype='object')
```

[39] 0s
```
cat = ['gender', 'SeniorCitizen', 'Partner', 'Dependents',
       'PhoneService', 'MultipleLines', 'InternetService',
       'OnlineSecurity', 'OnlineBackup', 'DeviceProtection', 'TechSupport',
       'StreamingTV', 'StreamingMovies', 'Contract', 'PaperlessBilling',
       'PaymentMethod', 'Churn']
```

[40] 0s
```
for i in cat:
  print(f"Uniuq values in {i} is : -")
  print(df[i].unique(), "\n")
```
```
Uniuq values in gender is : -
['Female' 'Male']

Uniuq values in SeniorCitizen is : -
[0 1]

Uniuq values in Partner is : -
['Yes' 'No']

Uniuq values in Dependents is : -
['No' 'Yes']

Uniuq values in PhoneService is : -
['No' 'Yes']

Uniuq values in MultipleLines is : -
['No phone service' 'No' 'Yes']

Uniuq values in InternetService is : -
['DSL' 'Fiber optic' 'No']

Uniuq values in OnlineSecurity is : -
['No' 'Yes' 'No internet service']

Uniuq values in OnlineBackup is : -
['Yes' 'No' 'No internet service']

Uniuq values in DeviceProtection is : -
['No' 'Yes' 'No internet service']

Uniuq values in TechSupport is : -
['No' 'Yes' 'No internet service']

Uniuq values in StreamingTV is : -
['No' 'Yes' 'No internet service']

Uniuq values in StreamingMovies is : -
['No' 'Yes' 'No internet service']

Uniuq values in Contract is : -
['Month-to-month' 'One year' 'Two year']

Uniuq values in PaperlessBilling is : -
['Yes' 'No']

Uniuq values in PaymentMethod is : -
['Electronic check' 'Mailed check' 'Bank transfer (automatic)'
 'Credit card (automatic)']

Uniuq values in Churn is : -
['No' 'Yes']
```

[41] 0s
```
# for feature engg. we alredy have lot much columns
```

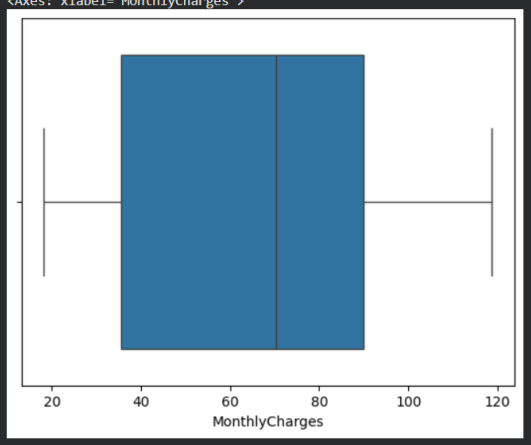
[43] ✓ 0s
```
# go for the eda
```

[44] ✓ 0s
```
# numerical features
```

[45] ✓ 0s
```
# outlier for the numerical feature
```

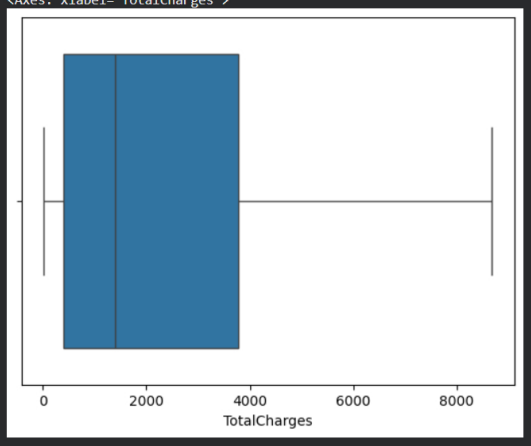
[46] ✓ 0s
```
sns.boxplot(data= df, x="MonthlyCharges")
```

```
<Axes: xlabel='MonthlyCharges'>
```



[47] ✓ 0s
```
sns.boxplot(data= df, x="TotalCharges")
```

```
<Axes: xlabel='TotalCharges'>
```



[48] ✓ 0s
```
# so no extra charrges for total as well as for monthly charges from the customers"
```

[49] ✓ 0s
```
df[["TotalCharges", "MonthlyCharges", "tenure"]].describe()
```

	TotalCharges	MonthlyCharges	tenure
count	7043.000000	7043.000000	7043.000000
mean	2281.916928	64.761692	32.371149
std	2265.270398	30.090047	24.559481
min	18.800000	18.250000	0.000000
25%	402.225000	35.500000	9.000000
50%	1397.475000	70.350000	29.000000
75%	3786.600000	89.850000	55.000000
max	8684.800000	118.750000	72.000000

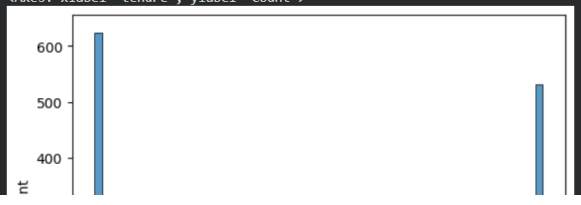
[50] ✓ 0s
```
# we have total 6 year od the tenure
```

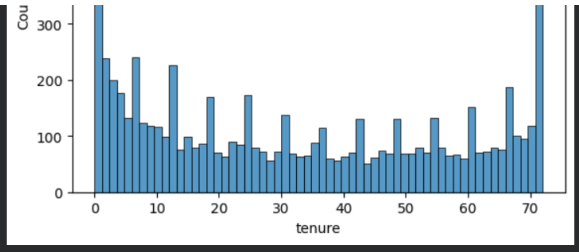
[51] ✓ 0s
```
# there is no outlier
```

[52] ✓ 0s
```
# tenure distribution
```

[54] ✓ 0s
```
sns.histplot(data= df, x="tenure", bins= 60 , )
```

```
<Axes: xlabel='tenure', ylabel='Count'>
```

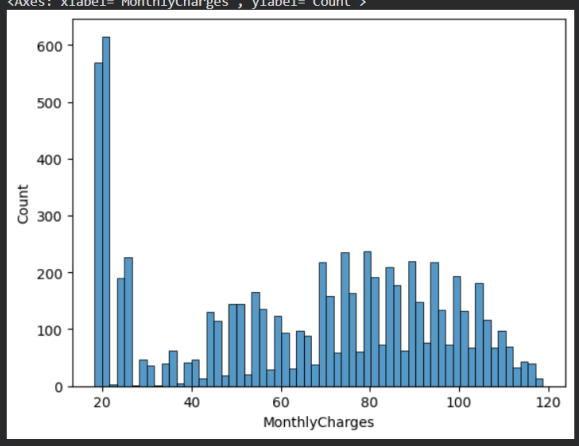




[55] ✓ 0s

```
sns.histplot(data= df, x="MonthlyCharges", bins=60)
```

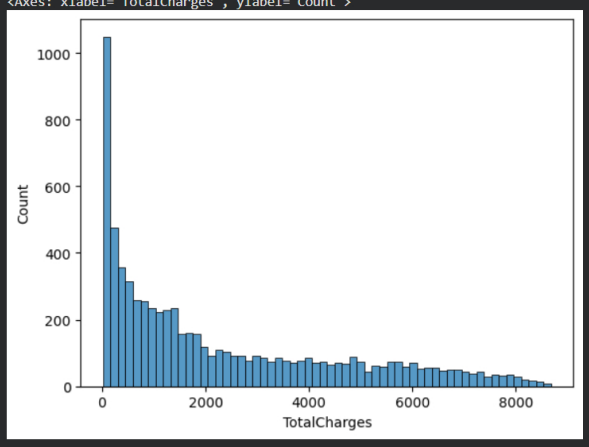
```
<Axes: xlabel='MonthlyCharges', ylabel='Count'>
```



[56] ✓ 0s

```
sns.histplot(data= df, x="TotalCharges", bins=60)
```

```
<Axes: xlabel='TotalCharges', ylabel='Count'>
```



[57] ✓ 0s

```
# all about the categorical data
```

[58] ✓ 0s

```
# contract
```

[59] ✓ 0s

```
df["Contract"].unique()
```

```
array(['Month-to-month', 'One year', 'Two year'], dtype=object)
```

[60] ✓ 0s

```
df["Contract"].value_counts()
```

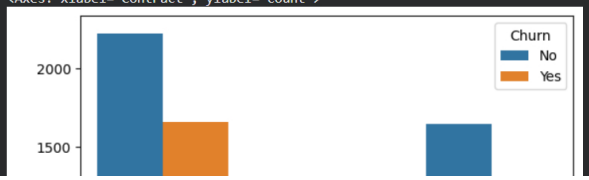
Contract	count
Month-to-month	3875
Two year	1695
One year	1473

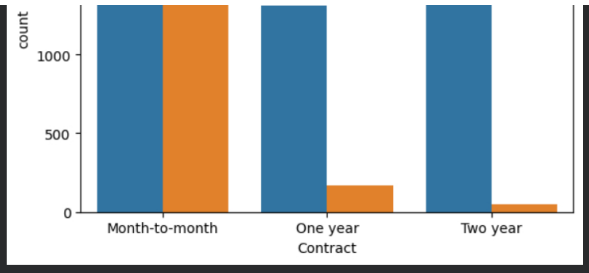
dtype: int64

[61] ✓ 0s

```
sns.countplot(data= df, x="Contract", hue="Churn")
```

```
<Axes: xlabel='Contract', ylabel='count'>
```





[63]
```
df.groupby("Contract")["Churn"].value_counts().unstack()
```

Contract	Churn	No	Yes
Month-to-month		2220	1655
One year		1307	166
Two year		1647	48

[64]
```
# Month to month customers majorly churned
```

[65]
```
# customers on month to month contract may neeedloyalty discount
# offer or incentive like give the service free
```

[66]
```
cat
```

```
['gender',
 'SeniorCitizen',
 'Partner',
 'Dependents',
 'PhoneService',
 'MultipleLines',
 'InternetService',
 'OnlineSecurity',
 'OnlineBackup',
 'DeviceProtection',
 'TechSupport',
 'StreamingTV',
 'StreamingMovies',
 'Contract',
 'PaperlessBilling',
 'PaymentMethod',
 'Churn']
```

[67]
```
df["gender"].unique()
```

```
array(['Female', 'Male'], dtype=object)
```

[68]
```
df["gender"].value_counts()
```

gender	count
Male	3555
Female	3488

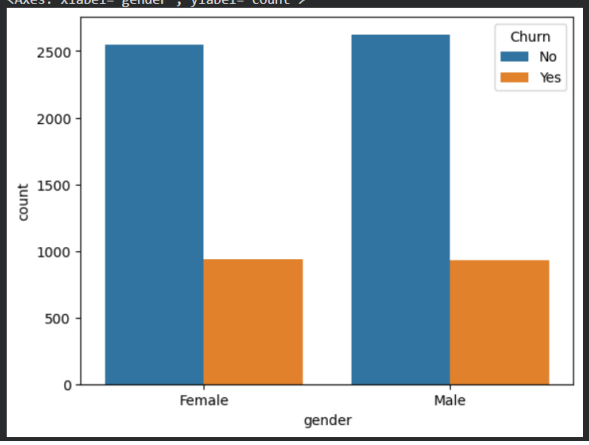
dtype: int64

[69]
```
df.groupby("gender")["Churn"].value_counts().unstack()
```

gender	Churn	No	Yes
Female		2549	939
Male		2625	930

[70]
```
sns.countplot(data= df, x="gender", hue="Churn")
```

```
<Axes: xlabel='gender', ylabel='count'>
```



[71] ✓ 0s

```
df.groupby("gender")["Churn"].value_counts(normalize=True).unstack()
```

Churn	No	Yes
gender		
Female	0.730791	0.269209
Male	0.738397	0.261603

[72] ✓ 0s

```
#next
```

[ ]

```
Start coding or generate with AI.
```

Colab paid products - Cancel contracts here

Variables Terminal ✓ 11:45 AM Python 3